



Databases

Andrzej Nowicki

CERN School of Computing, Chile 2026

Lecture Outline

- Lecture I – past, present, future
- Lecture II – schema design, SQL**
- Lecture III – scalability & reliability challenges



Our Challenge for Today

Imagine you're a Technical Manager of the CSC.

You are asked to keep track of the school editions, participants, locations.

The school had ~80 editions, ~5000 participants.

How do you start?

Remember what I said before:

There is no “one-size fits all” solution. Choose your databases wisely!

Document database

You could use a Document database – each school is a separate document.

```
{  
  "name": "CSC Chile",  
  "location": "Santiago, Chile",  
  "year": 2026,  
  "participants": ["AN", "KG", "AP", ...]  
}
```

```
{  
  "name": "CSC Villa Monastero",  
  "location": "Varenna, Italy",  
  "year": 1970  
}
```

How to design a schema for such a use-case?

Get as much information about your data:

- Informations you want to keep
- Data types of the columns

Try to avoid duplicating same information, but sometimes you can't avoid it.

ID	Name	Year	Location
1	tCSC on Security	2023	Split, HR
2	tCSC on ML	2024	Split, HR
3	CSC Latin America	2026	Santiago, CL

VS

ID	Name	Year	Location_ID
1	tCSC on Security	2023	1
2	tCSC on ML	2024	1
3	CSC Latin America	2026	2

ID	Location
1	Split, HR
2	Santiago, CL

How to design a schema for such a use-case?

What about participants?

ID	Name	Year	Location_ID
1	tCSC on Security	2023	1
2	tCSC on ML	2024	1
3	CSC Latin America	2026	2

ID	Name	Function	School_ID
100	Andrzej Nowicki	TM	1
101	Andrzej Nowicki	TM	2
102	Andrzej Nowicki	TM, LE	3



How to design a schema for such a use-case?

What about participants?

ID	Name	Year	Location_ID
1	tCSC on Security	2023	1
2	tCSC on ML	2024	1
3	CSC Latin America	2026	2

School_ID	Person_ID	Function
1	100	TM
2	100	TM
3	100	TM
3	100	LE

ID	Name
100	Andrzej Nowicki
101	Alberto Pace
102	Kristina Gunne

How to design a schema for such a use-case?

What if you want to keep track of affiliations?

ID	Name	Year	Location_ID
1	tCSC on Security	2023	1
2	tCSC on ML	2024	1
3	CSC Latin America	2026	2
9	CSC Moon	2040	

School_ID	Person_ID	Function
1	100	TM
2	100	TM
3	100	TM
3	100	LE
9	100	LE

ID	Name	Affiliation
100	Andrzej Nowicki	CERN
101	Alberto Pace	CERN
102	Kristina Gunne	CERN

Are you sure that my Affiliation will not change?

Are you sure that I won't get married and change my name?
... and get divorced and change my name again?

Should you keep track of that?

Database Normalization



Normalization is the process of structuring a relational database in order to:

- **Reduce data redundancy**
- **Prevent anomalies**
- **Improve data integrity, have coherent tables**

Over-normalization might hurt performance
De-normalization is sometimes used for better performance

1NF – all values are atomic, no duplicate rows, each row needs a unique primary key
(e.g. value of „LE , TM” from previous slide breaks this rule as it's not atomic)

2NF – 1NF + data depends on entire primary key: *(this rule only applies for tables with composite primary keys)*
For a composite key (StudentID, SchoolID), if the "StudentName" depends only on "StudentID" and not on the entire key, it violates 2NF. To normalize, move StudentName into a separate table where it depends only on "StudentID".

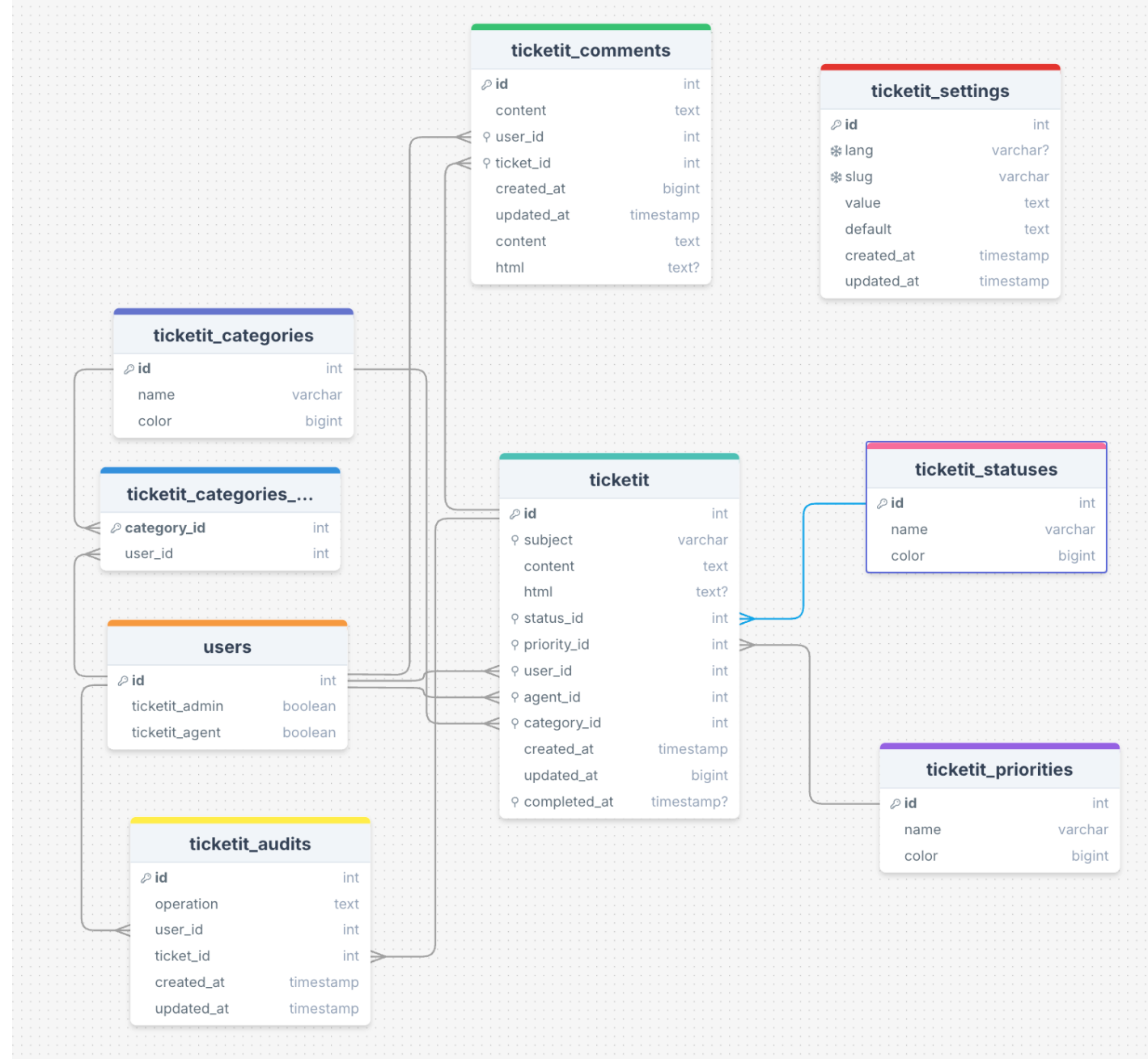
3NF – 2NF + no transitive dependencies

Table with (StudentID, SchoolID, Instructor). If Instructor depends on "SchoolID", and "SchoolID" depends on "StudentID", then Instructor indirectly depends on "StudentID", which violates 3NF. To resolve this, place Instructor in a separate table linked by "SchoolID".

How to design a schema?

ERD diagrams!

Keeps track of cardinality, optional/required relations, etc.



SQR


```
CREATE TABLE table_name(...);
DROP TABLE table_name;
ALTER TABLE table_name ADD COLUMN ...;
INSERT INTO table_name (col1) values (<val1>);
UPDATE table_name SET col = val WHERE <condition>;
DELETE FROM table_name WHERE <condition>;
SELECT * FROM table_name ORDER BY col [ASC|DESC];
SELECT * FROM table_name WHERE condition;
SELECT col, COUNT(*) FROM table_name GROUP BY col;
SELECT ... FROM table1 JOIN table2 ON table1.id = table2.id;
SELECT * FROM table WHERE id IN (SELECT id FROM ...);
CREATE INDEX idx_name ON table_name (column);
DROP INDEX idx_name;
BEGIN; UPDATE ...; COMMIT / ROLLBACK;
GRANT USAGE ON SCHEMA schema_name TO username;
GRANT SELECT ON table_name TO username;
GRANT ALL PRIVILEGES ON table_name TO username;
```


SQL – Structured Query Language



SQL SYNTAX
IS NOT ON
THE EXAM

SQL consists of:

- data definition language (DDL)

```
CREATE TABLE schools (school_id SERIAL PRIMARY KEY,  
                        name VARCHAR(20),  
                        year INTEGER);
```

- data manipulation language (DML)

```
INSERT INTO schools (name, year) VALUES ('CSC Latin America', 2026);
```

- data control language (DCL)

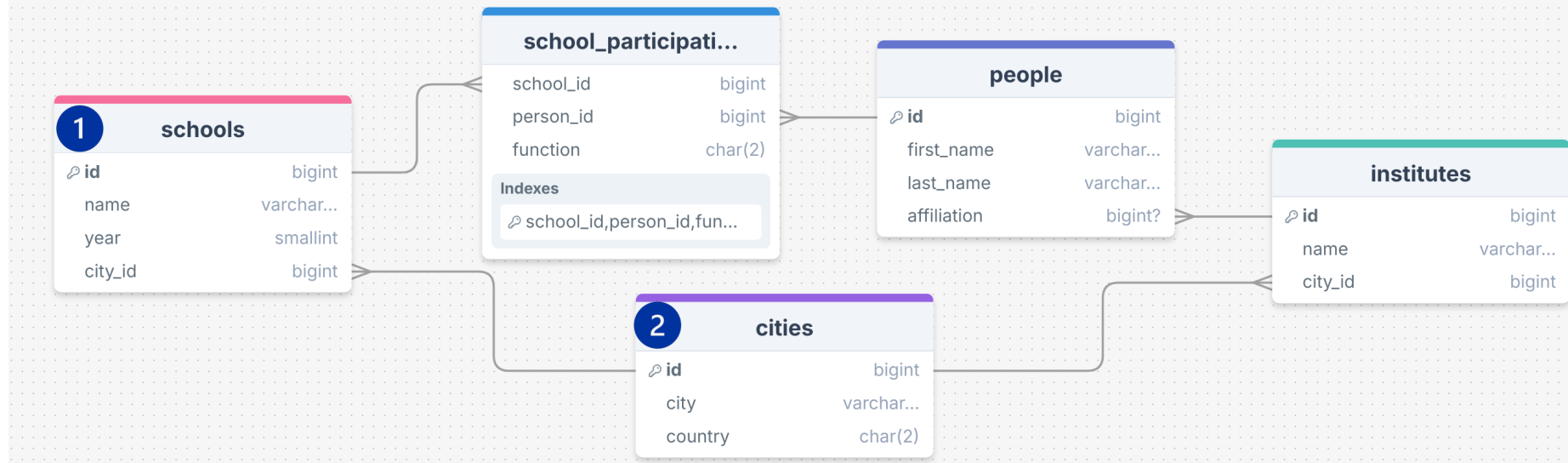
```
GRANT SELECT ON schools TO public;
```

- data query language (DQL)

```
SELECT school_id, name FROM schools WHERE year = 2026;
```

Beware! It's a “standard”, but each DB has it's own implementation differences. It's not fully interoperable!

Our schema

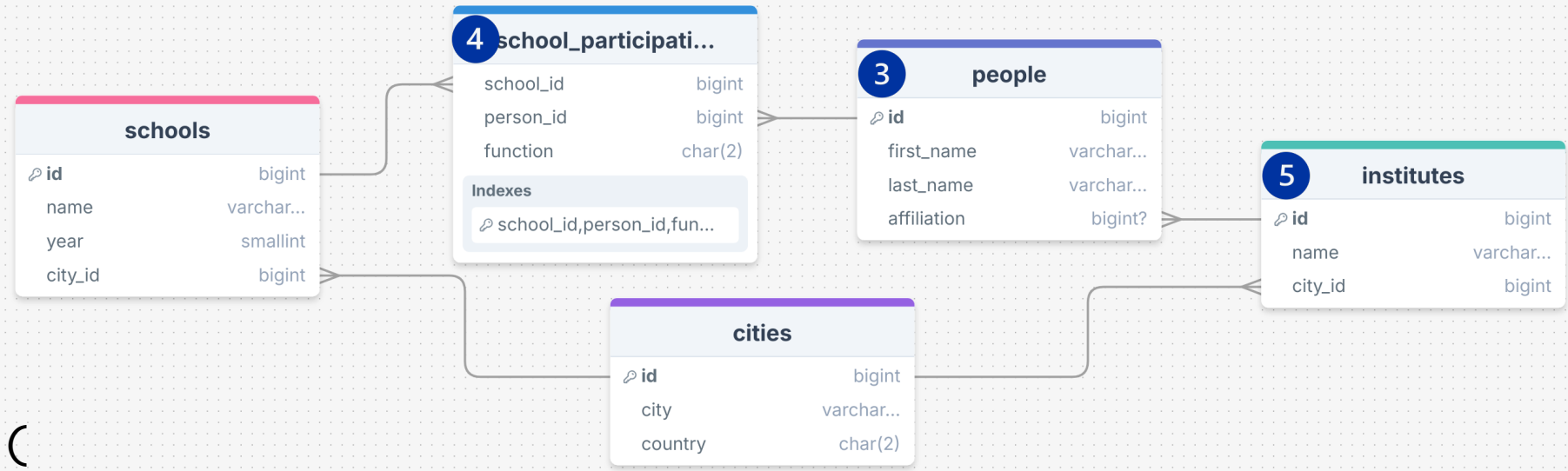


- 1

```
CREATE TABLE "schools"(  
  "id" INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
  "name" VARCHAR(255) NOT NULL,  
  "year" SMALLINT NOT NULL,  
  "city_id" INTEGER NOT NULL  
);
```
- 2

```
CREATE TABLE "cities"(  
  "id" INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
  "city" VARCHAR(255) NOT NULL,  
  "country" CHAR(2) NOT NULL  
);
```

Our schema



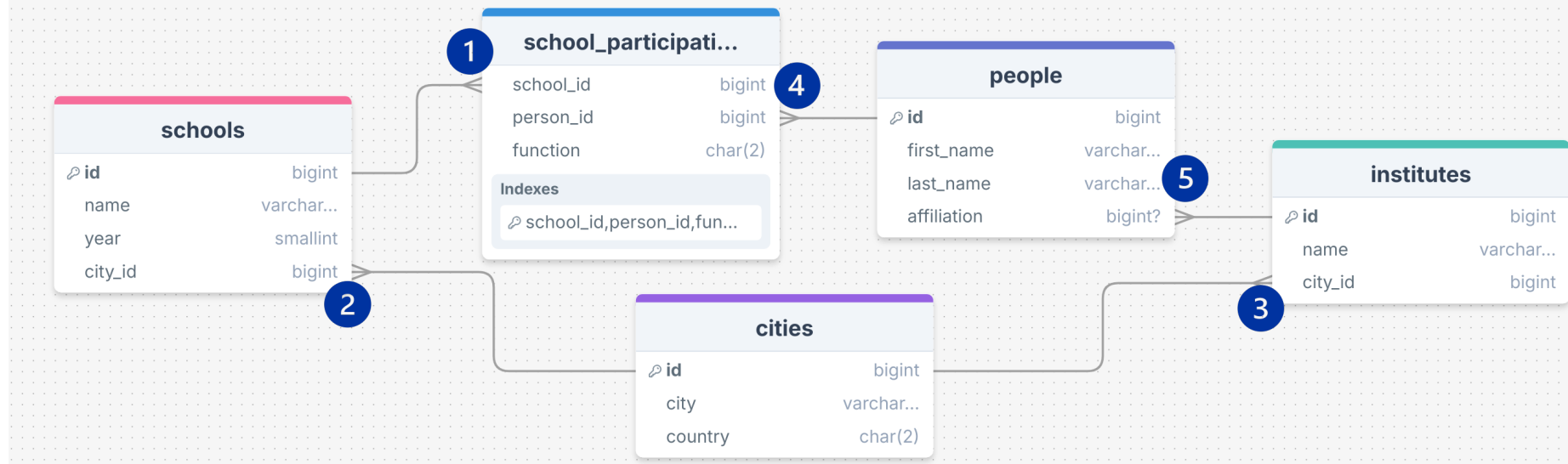
```
3 CREATE TABLE "people"(  
    "id" INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    "first_name" VARCHAR(255) NOT NULL,  
    "last_name" VARCHAR(255) NOT NULL,  
    "affiliation" INTEGER  
);
```

```
4 CREATE TABLE "school_participation"(  
    "school_id" INTEGER NOT NULL,  
    "person_id" INTEGER NOT NULL,  
    "function" CHAR(2) NOT NULL  
);  
ALTER TABLE
```

```
5 CREATE TABLE "institutes"(  
    "id" INTEGER PRIMARY KEY  
    GENERATED ALWAYS AS IDENTITY,  
    "name" VARCHAR(255) NOT NULL,  
    "city_id" INTEGER NOT NULL  
);
```

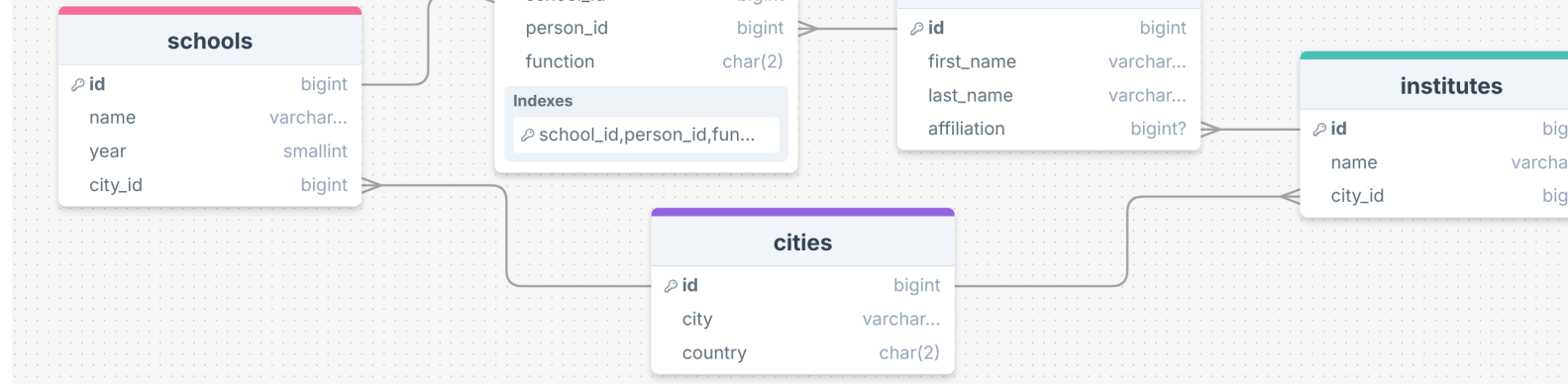
```
"school_participation" ADD PRIMARY KEY("school_id", "person_id", "function");
```

Our schema



- 1 `ALTER TABLE "school_participation" ADD CONSTRAINT "school_participation_school_id_foreign" FOREIGN KEY("school_id") REFERENCES "schools"("id");`
- 2 `ALTER TABLE "schools" ADD CONSTRAINT "schools_city_id_foreign" FOREIGN KEY("city_id") REFERENCES "cities"("id");`
- 3 `ALTER TABLE "institutes" ADD CONSTRAINT "institutes_city_id_foreign" FOREIGN KEY("city_id") REFERENCES "cities"("id");`
- 4 `ALTER TABLE "school_participation" ADD CONSTRAINT "school_participation_person_id_foreign" FOREIGN KEY("person_id") REFERENCES "people"("id");`
- 5 `ALTER TABLE "people" ADD CONSTRAINT "people_affiliation_foreign" FOREIGN KEY("affiliation") REFERENCES "institutes"("id");`

Let's add data



```
# INSERT INTO schools (name, year) VALUES ('CSC Latin America',2026);
ERROR: null value in column "city_id" of relation "schools" violates not-null constraint
DETAIL: Failing row contains (1, CSC Latin America, 2026, null).
```

```
# INSERT INTO cities (city, country) VALUES ('Santiago', 'CL');
```

```
INSERT 0 1
```

```
# SELECT * FROM cities;
```

```
 id |  city  | country
----+-----+-----
  1 | Santiago | CL
```

```
# INSERT INTO schools (name, year, city_id) VALUES ('CSC Latin America',2026,1);
```

```
INSERT 0 1
```

```
# SELECT * FROM schools;
```

```
 id |      name      | year | city_id
----+-----+-----+-----
  2 | CSC Latin America | 2026 |      1
```

```

# INSERT INTO schools (name, year, city_id) VALUES ('CSC Moon',2040,100);
ERROR: insert or update on table "schools" violates foreign key constraint "schools_city_id_foreign"
DETAIL: Key (city_id)=(100) is not present in table "cities".
# INSERT INTO cities (city, country) VALUES ('Santiago','CL');
INSERT 0 1
# SELECT * FROM cities;
 id |   city   | country
-----+-----+-----
  1 | Santiago | CL
  2 | Santiago | CL
# CREATE UNIQUE INDEX unique_cities ON cities(city, country);
ERROR: could not create unique index "unique_cities"
DETAIL: Key (city, country)=(Santiago, CL) is duplicated.
# DELETE FROM cities WHERE id=2;
DELETE 1
# CREATE UNIQUE INDEX unique_cities ON cities(city, country);
CREATE INDEX
# INSERT INTO cities (city, country) VALUES ('Santiago','CL');
ERROR: duplicate key value violates unique constraint "unique_cities"
DETAIL: Key (city, country)=(Santiago, CL) already exists.

```


SELECT QUERIES


```
# SELECT first_name, last_name
  FROM people
 WHERE last_name='Nowicki';
first_name | last_name
```

List people who's last name is Nowicki

```
-----+-----
Andrzej    | Nowicki
```

```
# SELECT *
  FROM people
 WHERE affiliation = ( SELECT id
                       FROM institutes
                       WHERE name='CERN' );
```

List people affiliated to CERN

```
id | first_name | last_name | affiliation
---+-----+-----+-----
 3 | Andrzej    | Nowicki   |          3
 4 | Kristina   | Gunne     |          3
 5 | Alberto    | Pace      |          3
```

```
# SELECT year, COUNT(1) how_many_schools
  FROM schools
 GROUP BY year;
```

How many schools were organised each year

```
year | how_many_schools
-----+-----
2026 |                2
```

```
# SELECT COUNT(DISTINCT person_id)
  FROM school_participation
 WHERE school_id = ( SELECT id
                    FROM schools
                    WHERE name='CSC Latin America' );
```

How many persons are participating in CSC Latin America?

Don't count people having 2 roles as 2 people

count

3

```
# SELECT s.name, s.year, c.city, c.country
  FROM schools s
 JOIN cities c ON s.city_id = c.id;
```

List schools information, include city & countries

name	year	city	country
CSC Latin America	2026	Santiago	CL
inverted CSC	2026	Geneva	CH

```
# SELECT s.name, s.year,
  (SELECT COUNT(DISTINCT sp.person_id)
   FROM school_participation sp
   WHERE sp.school_id = s.id)
  FROM schools s;
```

List schools with their number of participants
Don't count people having 2 roles as 2 people

name	year	count
CSC Latin America	2026	3
inverted CSC	2026	0

Quiz: Which query will return cities without schools

- 1 `SELECT *`
`FROM cities`
`WHERE id NOT IN (SELECT city_id FROM schools);`
- 2 `SELECT *`
`FROM cities`
`WHERE id IN (SELECT id FROM cities EXCEPT SELECT city_id FROM schools);`
- 3 `SELECT *`
`FROM cities c`
`WHERE (SELECT count(1)`
`FROM schools`
`WHERE city_id=c.id) = 0;`

set operations:
UNION, INTERSECT, EXCEPT

All 3 will provide the same results.

All 5 will provide the same results.

- 1 `SELECT *
FROM cities
WHERE id NOT IN (SELECT city_id FROM schools);`
- 2 `SELECT *
FROM cities
WHERE id IN (SELECT id FROM cities EXCEPT SELECT city_id FROM schools);`
- 3 `SELECT *
FROM cities c
WHERE (SELECT count(1)
FROM schools
WHERE city_id=c.id) = 0;`
- 4 `SELECT c.id, city, country
FROM cities c
LEFT JOIN schools s ON c.id=s.city_id
WHERE s.id IS NULL;`
- 5 `SELECT *
FROM cities c
WHERE NOT EXISTS (SELECT 1 FROM schools WHERE c.id = city_id);`

All 5 will provide the same results.

```
csc=# SELECT count(*) FROM cities WHERE id NOT IN (SELECT city_id FROM schools);
count
```

24099

Time: 9.404 ms

```
csc=# SELECT count(*) FROM cities WHERE id IN (SELECT id FROM cities EXCEPT SELECT city_id FROM schools);
count
```

24099

Time: 24.971 ms

```
csc=# SELECT count(*) FROM cities c WHERE c.id = (SELECT city_id FROM schools WHERE city_id = c.id);
count
```

24099

Time: 21.675 ms

```
csc=# SELECT count(*) FROM cities c LEFT JOIN schools s ON c.id=s.city_id WHERE s.id IS NULL;
count
```

24099

Time: 2.191 ms

```
csc=# SELECT count(*) FROM cities c WHERE NOT EXISTS (SELECT 1 FROM schools WHERE c.id = city_id);
count
```

24099

Time: 1.395 ms

```
csc=# EXPLAIN SELECT count(*) FROM cities WHERE id NOT IN (SELECT city_id FROM schools);
QUERY PLAN
```

```
-----
Aggregate (cost=480.41..480.42 rows=1 width=8)
-> Seq Scan on cities (cost=1.02..450.29 rows=12050 width=0)
    Filter: (NOT (ANY (id = (hashed SubPlan 1).col1)))
    SubPlan 1
        -> Seq Scan on schools (cost=0.00..1.02 rows=2 width=4)
```

```
csc=# EXPLAIN SELECT count(*) FROM cities WHERE id IN (SELECT id FROM cities EXCEPT SELECT city_id FROM schools);
QUERY PLAN
```

```
-----
Aggregate (cost=1625.63..1625.64 rows=1 width=8)
-> Hash Join (cost=690.27..1565.38 rows=24101 width=0)
    Hash Cond: ("ANY_subquery".id = cities.id)
    -> Subquery Scan on "ANY_subquery" (cost=0.00..811.83 rows=24101 width=4)
        -> HashSetOp Except (cost=0.00..811.83 rows=24101 width=8)
            -> Append (cost=0.00..751.57 rows=24103 width=8)
                -> Subquery Scan on "SELECT* 1" (cost=0.00..630.02 rows=24101 width=8)
                    -> Seq Scan on cities cities_1 (cost=0.00..389.01 rows=24101 width=4)
                -> Subquery Scan on "SELECT* 2" (cost=0.00..1.04 rows=2 width=8)
                    -> Seq Scan on schools (cost=0.00..1.02 rows=2 width=4)
            -> Hash (cost=389.01..389.01 rows=24101 width=4)
                -> Seq Scan on cities (cost=0.00..389.01 rows=24101 width=4)
```

```
csc=# EXPLAIN SELECT count(*) FROM cities c WHERE (SELECT count(1) FROM schools WHERE city_id=c.id ) = 0;
QUERY PLAN
```

```
-----
Aggregate (cost=25454.35..25454.36 rows=1 width=8)
-> Seq Scan on cities c (cost=0.00..25454.05 rows=121 width=0)
    Filter: ((SubPlan 1) = 0)
    SubPlan 1
        -> Aggregate (cost=1.03..1.04 rows=1 width=8)
            -> Seq Scan on schools (cost=0.00..1.02 rows=1 width=0)
                Filter: (city_id = c.id)
```

```
csc=# EXPLAIN SELECT count(*) FROM cities c LEFT JOIN schools s ON c.id=s.city_id WHERE s.id IS NULL;
QUERY PLAN
```

```
-----
Aggregate (cost=480.76..480.77 rows=1 width=8)
-> Hash Left Join (cost=1.04..480.45 rows=12050 width=0)
    Hash Cond: (c.id = s.city_id)
    Filter: (s.id IS NULL)
    -> Seq Scan on cities c (cost=0.00..389.01 rows=24101 width=4)
    -> Hash (cost=1.02..1.02 rows=2 width=8)
        -> Seq Scan on schools s (cost=0.00..1.02 rows=2 width=8)
```

```
csc=# EXPLAIN SELECT count(*) FROM cities c WHERE NOT EXISTS (SELECT 1 FROM schools WHERE c.id = city_id);
QUERY PLAN
```

```
-----
Aggregate (cost=754.56..754.57 rows=1 width=8)
-> Hash Anti Join (cost=1.04..694.31 rows=24099 width=0)
    Hash Cond: (c.id = schools.city_id)
    -> Seq Scan on cities c (cost=0.00..389.01 rows=24101 width=4)
    -> Hash (cost=1.02..1.02 rows=2 width=4)
        -> Seq Scan on schools (cost=0.00..1.02 rows=2 width=4)
```

**There is more than one way to get your result.
The DB will try to find the optimal one.**



The citites table populated with 24101 rows

```

[ WITH [ RECURSIVE ] with_query [, ...] ]
SELECT [ ALL | DISTINCT [ ON ( expression [, ...] ) ] ]
    [ { * | expression [ [ AS ] output_name ] } [, ...] ]
    [ FROM from_item [, ...] ]
    [ WHERE condition ]
    [ GROUP BY [ ALL | DISTINCT ] grouping_element [, ...] ]
    [ HAVING condition ]
    [ WINDOW window_name AS ( window_definition ) [, ...] ]
    [ { UNION | INTERSECT | EXCEPT } [ ALL | DISTINCT ] select ]
    [ ORDER BY expression [ ASC | DESC | USING operator ] [ NULLS { FIRST | LAST } ] [, ...] ]
    [ LIMIT { count | ALL } ]
    [ OFFSET start [ ROW | ROWS ] ]
    [ FETCH { FIRST | NEXT } [ count ] { ROW | ROWS } { ONLY | WITH TIES } ]
    [ FOR { UPDATE | NO KEY UPDATE | SHARE | KEY SHARE } [ OF from_reference [, ...] ] [ NOWAIT | SKIP LOCKED ] [...] ]

```

where *from_item* can be one of:

```

[ ONLY ] table_name [ * ] [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
    [ TABLESAMPLE sampling_method ( argument [, ...] ) [ REPEATABLE ( seed ) ] ]
[ LATERAL ] ( select ) [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
with_query_name [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
[ LATERAL ] function_name ( [ argument [, ...] ] )
    [ WITH ORDINALITY ] [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
[ LATERAL ] function_name ( [ argument [, ...] ] ) [ AS ] alias ( column_definition [, ...] )
[ LATERAL ] function_name ( [ argument [, ...] ] ) AS ( column_definition [, ...] )
[ LATERAL ] ROWS FROM( function_name ( [ argument [, ...] ] ) [ AS ( column_definition [, ...] ) ] [, ...] )
    [ WITH ORDINALITY ] [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
from_item join_type from_item { ON join_condition | USING ( join_column [, ...] ) [ AS join_using_alias ] }
from_item NATURAL join_type from_item
from_item CROSS JOIN from_item

```

and *grouping_element* can be one of:

```

( )
expression
( expression [, ...] )
ROLLUP ( { expression | ( expression [, ...] ) } [, ...] )
CUBE ( { expression | ( expression [, ...] ) } [, ...] )
GROUPING SETS ( grouping_element [, ...] )

```

and *with_query* is:

```

with_query_name [ ( column_name [, ...] ) ] AS [ [ NOT ] MATERIALIZED ] ( select | values | insert | update | delete | merge )
    [ SEARCH { BREADTH | DEPTH } FIRST BY column_name [, ...] SET search_seq_col_name ]
    [ CYCLE column_name [, ...] SET cycle_mark_col_name [ TO cycle_mark_value DEFAULT cycle_mark_default ] USING cycle_path_col_name ]

```

TABLE [ONLY] *table_name* [*]

Trying to understand the SELECT...

1. All elements in the FROM clause are computed (JOINS between tables, etc.)
2. WHERE conditions are applied (if any)
3. (if there is a) GROUP BY clause (or aggregate functions), the output is combined into groups
 - a. if there's a HAVING clause, another filtering of the result
4. SELECT clause (expressions, columns) are calculated
 - a. if there's a DISTINCT clause, remove duplicates
5. ORDER BY clause is applied (result set is sorted)
6. LIMIT & OFFSET clause is applied (restricts the result set, useful for pagination)

```
SELECT  
[ { * | expression [ [ AS ] output_name ] }  
[ FROM from_item [, ...] ]  
[ WHERE condition ]  
[ GROUP BY [ ALL | DISTINCT ] grouping_element [, ...] ]  
[ HAVING condition ]  
[ ORDER BY expression [ ASC | DESC | USING operator ] [, ...] ]  
[ LIMIT { count | ALL } ]  
[ OFFSET start [ ROW | ROWS ] ]
```


Views

1. Virtual “tables” based on saved SQL queries
2. Simplify complex queries

```
# create view schools_per_year as
    select year, count(1)
    from schools
    group by year;
```

```
# select * from schools_per_year;
```

```
year | count
-----+-----
2026 |      2
```

Access Control

1. You can control access to tables:

GRANT SELECT, UPDATE, INSERT ON schools TO kristina;

GRANT DELETE ON schools TO andrzej;

GRANT DELETE ON people TO alberto;

2. You can also create roles which group permissions:

CREATE ROLE director_role;

GRANT DELETE ON people TO director_role;

GRANT director_role TO alberto;

Summary

- You should understand how database technologies evolved and get the concept of RDBMS
- You should know the basic concepts of schema design
- You should be able to write basic SQL queries

We will put those skills into practice during the exercise session